Microprocessor & Assembly Language

CSE 0611-3221

Introduction to IBM PC Assembly Language

Presented By

Saifur Rahman

Lecturer

Dept. of CSE

NBIU

References

Assembly Language Programming and Organization of the IBM PC

By Ytha Yu and Charles Maru

IBM PC

- IBM PC: International Business Machines Personal Computer
- Introduced in 1981.
- One of the first widely available computers designed for individual use.

Key Features

- **Microprocessor:** Intel 8088, 8-bit processor.
- **Operating System:** MS-DOS (command-line based).
- **Expandability:** Slots for custom expansion cards.
- **Compatibility:** Set the "PC compatible" standard.

Assembly Language

- **Assembly Language:** Translated into machine code by an assembler.
- **Instruction:** Each statement becomes machine code.
- **Assembler Directive:** Instructs tasks like memory allocation or procedure creation.
- Both instructions and directives have up to **four fields**:

Name	Operation	Operand (s)	Comment

Assembly Language (cont'd)

Instruction Example:

```
START: MOV CX, 5 ;initialize counter
```

- Label: START
- Operation: MOV
- Operands: CX, 5
- Comment: ;initialize counter

Name	Operation	Operand (s)	Comment
START	MOV	CX, 5	; initialize counter
MAIN	PROC		

Assembler Directive Example:

```
MAIN PROC
```

- Name: MAIN
- Operation: PROC
- Function: Creates a procedure named MAIN

Name Field

Used for labels, procedure names, and variables, translated into memory addresses.

Rules:

- Length: 1 to 31 characters.
- Allowed: Letters, digits, _, $, ?, @, . (period must be first if used).
- No embedded spaces or starting with a digit.
- Case-insensitive.

Legal Names:

- ***VAR1:*** Contains letters and digits, no spaces or illegal characters.
- ***_count:*** Starts with an underscore, which is allowed.
- ***$total:*** Starts with a dollar sign, a permitted special character.
- ***.loop:*** Starts with a period, which is allowed as the first character.
- ***result_123:*** Combines letters, digits, and underscores, which are all valid.

Name Field (cont'd)

Illegal Names:

- ***123abc:*** Starts with a digit, which is not allowed.
- ***my variable:*** Contains a space, and embedded blanks are not permitted.
- ***total@#:*** Contains special characters (#) not allowed in names.
- ***abc.:*** Ends with a period; periods can only be the first character.
- ***my-variable!:*** Includes an exclamation mark, which is not a permitted special character.

Reasons:

- Names can't start with digits.
- No embedded spaces.
- Only specific special characters are allowed: _, $, ?, @, and . (only as the first character).

Operation Field

Contains symbolic **operation codes (opcodes)** like MOV, ADD, SUB.
Examples:

- ***NOP:*** No operands, does nothing.
- ***INC AX:*** One operand, increments AX by 1.
- ***ADD WORD1, 2:*** Two operands, adds 2 to memory location WORD1.

Two-Operand Instructions:

- **First Operand:** Destination (register or memory where the result is stored).
- **Second Operand:** Source (value used in the operation).

Pseudo-Ops: Directives that are not translated into machine code. They instruct the assembler to perform specific tasks during the assembly process. Example: PROC

Numbers

Table 1: Numbers

Number	Type
11011	Decimal
11011B	Binary
64223	Decimal
-2134D	Decimal
1,234	Illegal (contains a non-digit character)
1B4DH	Hex
1B4D	Illegal (doesn’t end in H)
FFFFH	Illegal (doesn’t begin with a decimal sign)
0FFFFH	Hex

Characters

Enclosure: Use single (') or double (") quotes for characters and strings.

- Examples: 'A', "hello"

ASCII Translation: Characters are converted to ASCII codes by the assembler.

- Note: 'A' is equivalent to 41h (ASCII code for "A").

Table 2: Data Defining Pseudo-Ops

Number	Type
DB	Define Byte
DW	Define Word
DD	Define Double Word
DQ	Define Quad Word (Four Words)
DT	Define Ten Bytes

Variables

```
Name    Pseudo-Ops  Initial Value
ALPHA   DB          ? ; Define byte with not yet initialized
COUNT   DW          5  ; Define word with value 5
WRD     DW          -2 ; Define word with initial value -2
CHAR    DB          'A'  ; Define byte with character 'A'
NUM     DW          1000  ; Define word with value 1000
```

Arrays

Array is simply a sequence of memory bytes or words.

```
B_ARRAY   DB     10H, 20H, 30H
```

- The name B_ARRAY refers to the first byte.
- B_ARRAY + 1 refers to the second byte.
- B_ARRAY + 2 refers to the third byte.

Symbol	Address	Contents
B_ARRAY	200h	10h
B_ARRAY + 1	201h	20h
B_ARRAY + 2	202h	30h

Arrays (cont'd)

```
W_ARRAY    DW     1000, 40, 29887, 329
```

Symbol	Address	Contents
W_ARRAY	0300h	1000d
W_ARRAY + 2	0302h	40d
W_ARRAY + 4	0304h	29887d
W_ARRAY + 6	0306h	329d

```
WORD1      DW     1234H
```

WORD1	Low Byte	Contains 34H
WORD1 + 1	High Byte	Contains 12H

Strings

An array of ASCII codes can be initialized using a string of characters.

```
LETTERS        DB      'ABC'
```

Is equivalent to

```
LETTERS        DB      41H, 42H, 43H
```

```
MSG        DB      'HELLO', 0AH, 0DH, '$'
```

Is equivalent to

```
MSG        DB      48H, 45H, 4CH, 4CH, 4OH, 0AH, 0DH, 24H
```

; ASCII values for 'H', 'E', 'L', 'L', 'O', newline (0AH), carriage return (0DH), and '$'

Named Constant

EQU (equates): To assign a name to a constant

Format:

```
name        EQU        constant
```

Example:

```
LF          EQU        0AH
```

So,

```
MOV    DL, 0AH
MOV    DL, LF
```

Are similar.

Basic Instructions

- **MOV:** Transfer data between
  - Registers
  - A register and a memory location
  - To move a number directly into register or memory location.

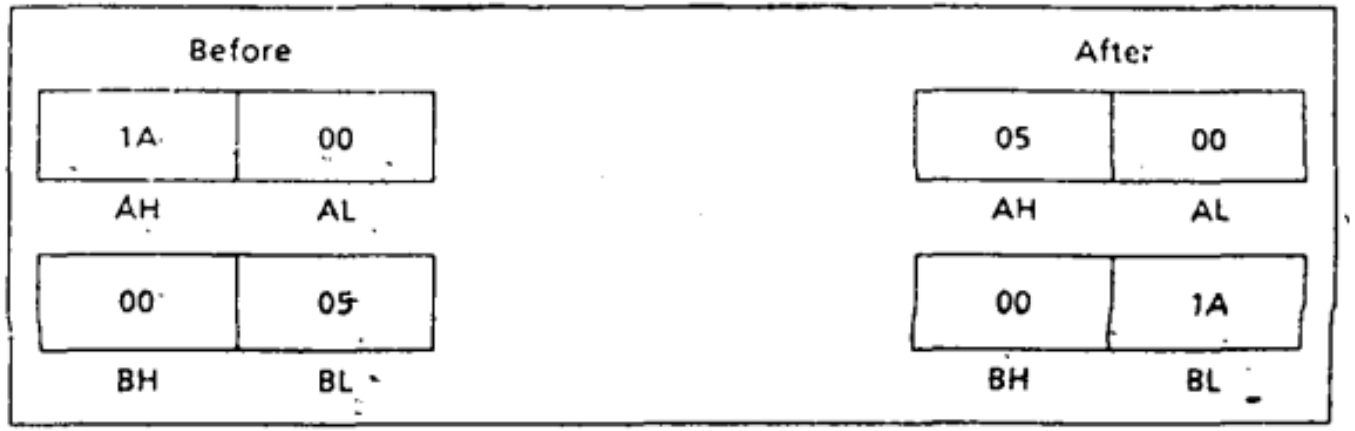
Format:

```
MOV     destination, source
```

- **XCHG:** Exchange the contents of
  - Two Registers
  - A register and a memory location

Format:

```
XCHG destination, source
XCHG AH, BL
```



Basic Instructions (cont'd)

Table 3: Legal Combination of Operands for MOV

Source Operand	Destination Operand			
	General Register	Segment Register	Memory Location	Constant
General Register	Yes	Yes	Yes	No
Segment Register	Yes	No	Yes	No
Memory Location	Yes	Yes	No	No
Constant	Yes	No	Yes	No

Table 4: Legal Combination of Operands for XCHG

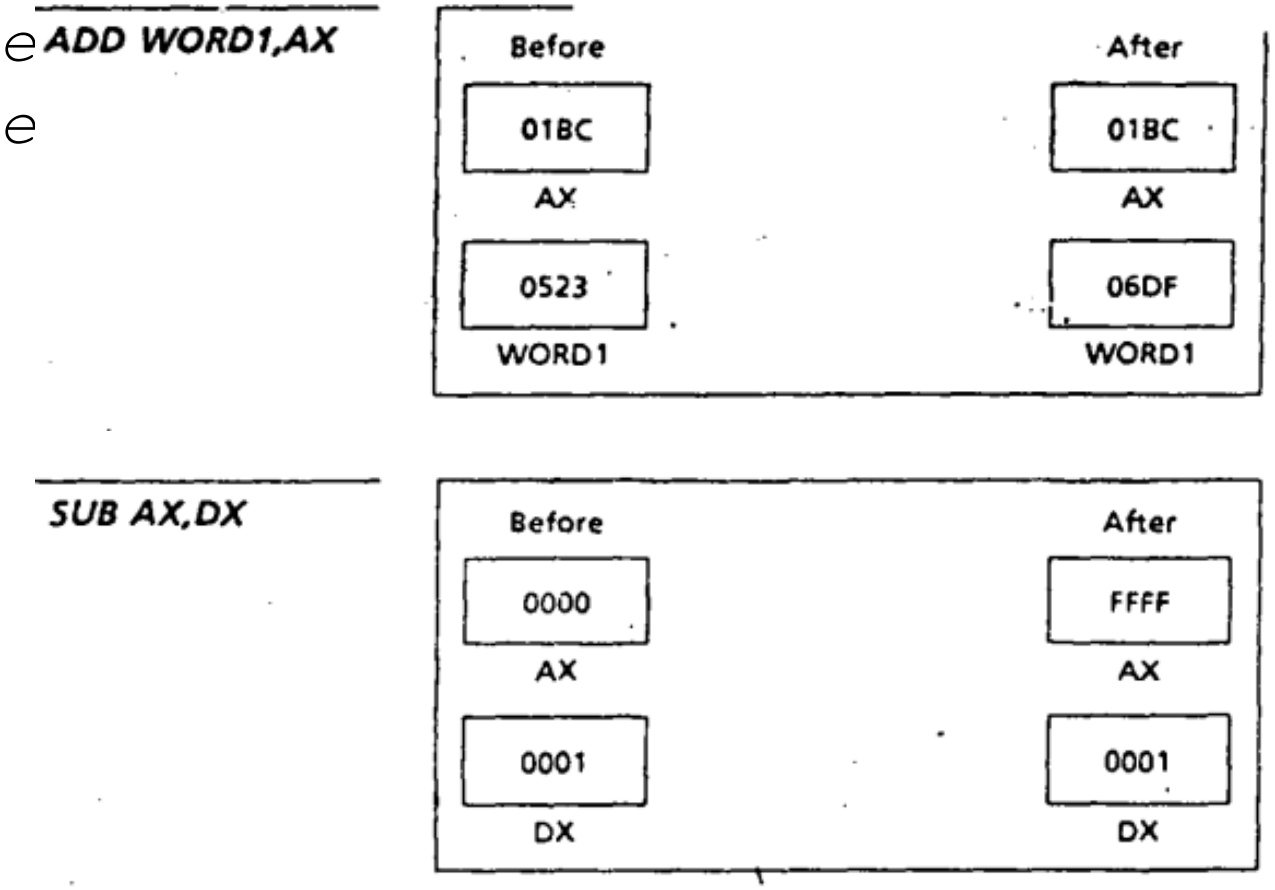
Source Operand	Destination Operand	
	General Register	Memory Location
General Register	Yes	Yes
Memory Location	Yes	No

Basic Instructions (cont'd)

```
ADD destination, source
SUB destination, source
```

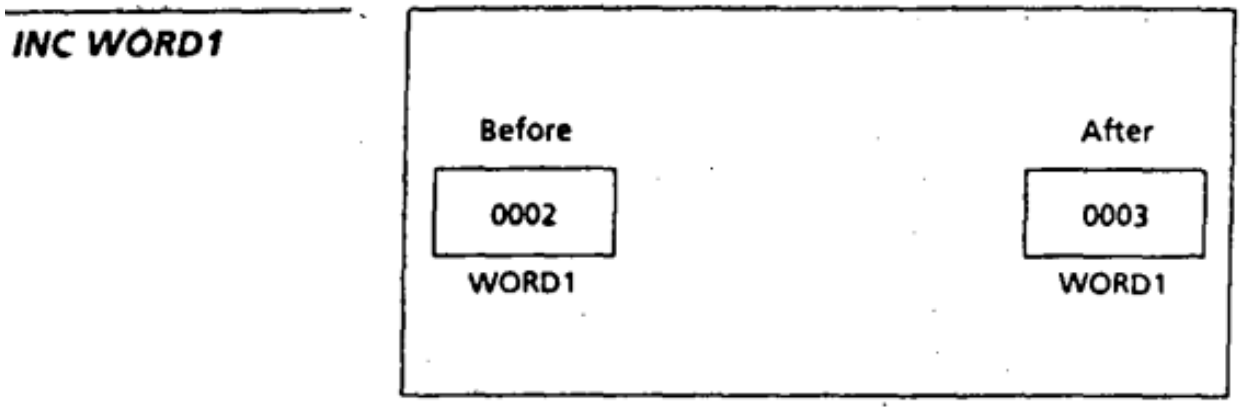
Table 5: Legal Combination of Operands for ADD and SUB

Source Operand	Destination Operand	
	General Register	Memory Location
General Register	Yes	Yes
Memory Location	Yes	No
Constant	Yes	Yes

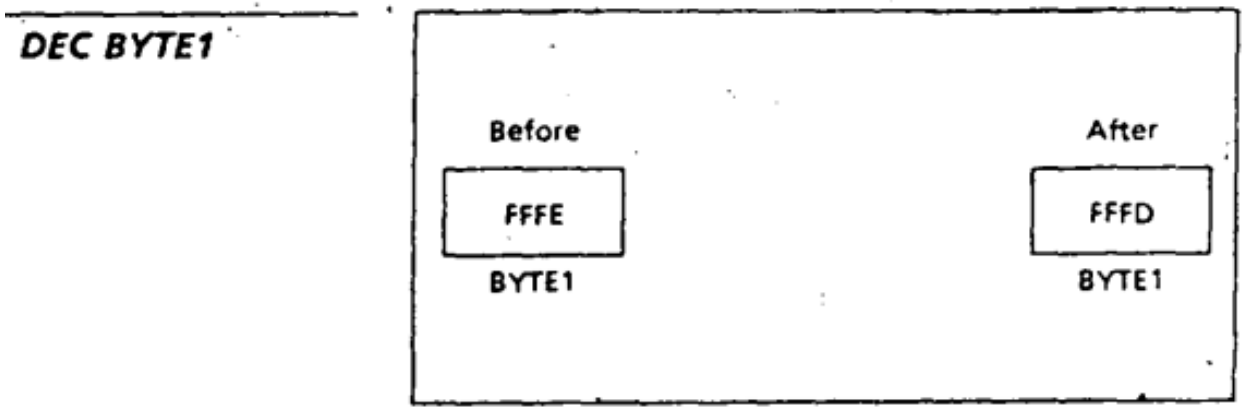


Basic Instructions (cont'd)

INC destination

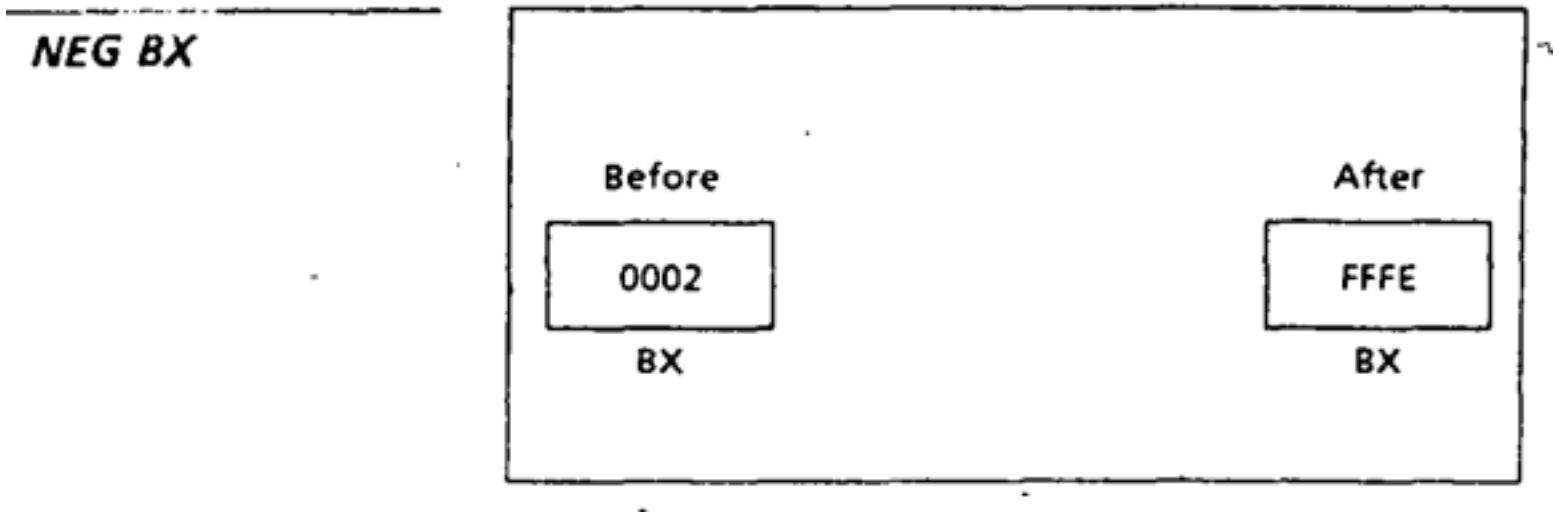


DEC destination



Basic Instructions (cont'd)

NEG destination



High-Level to Assembly Language Translation

Statement	**Translation**
`B = A`	`MOV AX, A ; move A into AX` `MOV B, AX ; and then into B`

`A = 5 - A`	`MOV AX, 5 ; put 5 in AX`
	`SUB AX, A ; AX contains 5 - A`
	`MOV A, AX ; put it back in A`
	
	`NEG A ; A = -A`
	`ADD A, 5 ; A = -A + 5 / A = 5 - A`

`A = B - 2 x A`	`MOV AX, B ; AX has B`
	`SUB AX, A ; AX has B - A`
	`SUB AX, A ; AX has B - 2 x A`
	`MOV A, AX ; move results to A`

Program Structure (Memory Models)

The size of a program's code and data is set using the **.MODEL** directive.

```
.MODEL          memory_model
```

Model	Description
SMALL	Code in one segment Data in one segment
MEDIUM	Code in more than one segment Data in one segment
COMPACT	Code in one segment Data in more than one segment
LARGE	Code in more than one segment Data in more than one segment No array larger than 64KB
HUGE	Code in more than one segment Data in more than one segment Arrays may be larger than 64KB

Program Structure (cont'd)

Data Segment

- Contains Variable Definitions.
- Constant Definitions are often made here.

```
.DATA
WORD    DW  5
MSG         DB  'We Hate Assembly'
MASK    EQU     10011010B
```

Stack Segment

- Sets aside a block of memory to store the stack.

```
.STACK          size
.STACK          100H
```

(If size is omitted, 1KB is set aside for the stack area)

Program Structure (cont'd)

Code Segment

- Contains Program Instruction
- Instructions are organized as **Procedures**. (Similar to functions in C)

```
.CODE        name
```

(Here **name** is optional)

Procedure Definition

```
name PROC
; body of procedure
name ENDP
```

Input and Output Instructions

- The CPU connects to external devices (**Peripherals**) through I/O ports using `**IN**` and `**OUT**` instructions for fast communication.
- However, most programs use **built-in service routines** instead because they are simpler and more compatible across different computers.
- There are two types of service routines:
  1. **BIOS routines:** Stored in the computer's memory, they handle **basic tasks** like controlling the screen.
  2. **DOS routines:** Handle more **complex tasks**, like printing, and rely on BIOS to manage the hardware.

The INT Instruction

- Invokes a **DOS** or **BIOS** routine.

```
INT        interrupt_number

INT        21H
```

- In assembly language, **interrupts** are used to call specific routines.
- **INT 16h** triggers a **BIOS routine** for keyboard input.
- **INT 21h** is used to access many **DOS** functions.
- To use it, a **function number** is placed in the **AH** register, and **INT 21h** is called.
- This allows programs to perform tasks like reading files or displaying output.

Input and Output Instructions (cont'd)

- **Single-key Input**

Input: AH = 1

Output: AL = ASCII code (if character key is pressed)
= 0 (if non-character key is pressed)

Function Number	Routine
1	Single-key Input
2	Single-character Output
9	Character String Output

```
MOV    AH, 1  ; input key function
INT    21h    ; ASCII Code in AL
```

Input and Output Instructions (cont'd)

- **Display a character**

Input: AH = 2
DL = ASCII code of display character or control character

Output: AL = ASCII code of display character or control character

Function Number	Routine
1	Single-key Input
2	Single-character Output
9	Character String Output

```
MOV    AH, 2  ; display character function
MOV    DL, '?'   ; character is '?'
INT    21h    ; display
```

Input and Output Instructions (cont'd)

- Function 2 can also perform control functions.
- The DL register holds the ASCII code of a control character, and when INT 21h is called, the corresponding control function is executed.
- Key control characters include:

ASCII Code (Hex)	Symbol	Function
7	BEL	Beep (sounds a tone)
8	BS	Backspace
9	HT	Tab
A	LF	Line feed (new line)
D	CR	Carriage Return (start of current line)

Input and Output Instructions (cont'd)

- **Display a String**

Input: AH = 9
DX = offset address of string

The String must end with a '$' character.

We use the LEA instruction for string output.

```
LEA        destination, source

LEA DX, MSG
MOV AH, 9
INT 21h
```

Function Number	Routine
1	Single-key Input
2	Single-character Output
9	Character String Output

Program Segment Prefix

- When a program is loaded into memory, DOS adds a 256-byte **Program Segment Prefix (PSP)**, which holds information about the program.
- DOS sets the **DS** and **ES** registers to the **segment number of this PSP**.
- This means that DS doesn't point to the data segment initially.
- To fix this and point DS to the actual data segment, a program begins with these two instructions:

1. **`MOV AX, @DATA`**
   - Loads the data segment address into the AX register.

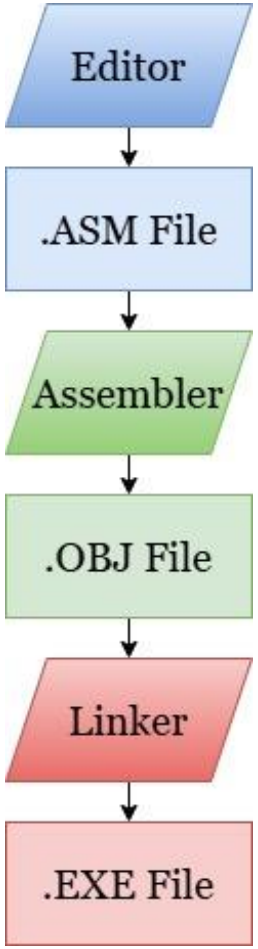
2. **`MOV DS, AX`**
   - Moves the value in AX into the DS register, setting DS to the correct data segment.

Programming Steps

Steps to Execute Assembly Code:

1. **Write the Source Code**
   - Use a text editor or word processor to create the assembly code (source file).

2. **Assemble the Code**
   - Use an assembler **(MASM: Microsoft Macro Assembler)** to translate the source code into machine language, creating an object file (.OBJ).

3. **Link the Object Files**
   - The LINK program combines one or more object files, fills in missing addresses, and creates an executable (.EXE) file.

4. **Run the Executable**
   - Load the executable file into memory and run it to execute the program.